

Individual Reflective Report

Autonomous Search and Rescue Drone

AENGM0074 — Group Project in Aerial Robotics

“An honest reflection on an 820-commit, solo-led build: what it delivered, what it cost, and what it taught me about team-first engineering. The mission flew in simulation. The lesson is that a lone sprint is not the same as a team project, and both have to be learned.”

Dmytro “Apollo” Chuprynyuk

Student ID: [TODO: user verify]

Team: *SAR Drone Team* | Role: Computer Vision & Autonomy Lead

Unit	AENGM0074 Group Project in Aerial Robotics
Deliverable	D7 — Individual Reflective Report
Supervisor	Dr. Arthur Richards [TODO: user verify]
Submission	Thursday, Week 23 — April 2026
Word count	~2 950 (body, excluding cover, refs, appendices)
Page count	5 (body) cover and rubric map excluded

I confirm that this report is my own work, that all sources are acknowledged, and that the reflections, judgements, and numerical claims made herein are traceable to commits, logs, and artefacts in the project repository. Co-authored artefacts are attributed to their contributors in the team report (D6).

Rubric Map

Question	Section	Key evidence	Rubric hit
Q1 My role	§1 My role	Commit 7eb7cc8 (02:14, 2026-03-31); drift from CV lead to infra owner; Hatton L4 reframe of “trust my hands over a conversation”; internship 48-h signal.	Specialist; Self-awareness
Q2 My team	§2 My team	Kickoff structure, silent wk16 drift, wk17 FSM conflict (DESIGN_DECISIONS.md:107–109), wk20 PM/complaint-letter conflict.	Team; Decision; M16.c
Q3 My impact	§3 My impact	Full Kolb cycle on sim-first pivot; 1411 → 754-line refactor, 146 pytests; 6-method comms A/B evaluation; sober sim-to-real limits.	Specialist; Comms; M17.d
Q4 My support	§4 My support	Edward (mavproxy), Robin (CENTERING timeout), Demetro (FDR register), pilot (jargon before/after + 4-button layout), Sid (DJI video).	Team; Self-awareness; M17.c

My role

At 02:14 on 2026-03-31, after the second cancelled field day, I pushed commit 7eb7cc8 — a fix for a double-scaling bug that had been quietly offsetting my GPS target estimates by more than 100 metres. I had been on the project for sixteen weeks by that point, and the commit was my 611th. What stopped me, staring at the diff, was not the bug itself but a smaller and more uncomfortable realisation: the file I was patching, `passive_watch.py`, was not in my original remit. Neither were the 127 unit tests in `tests/unit/`, the `main.py` refactor from 1411 to 754 lines, the `simple_simulator.py` MVP, or the D6 bibliography I would rebuild two days later. I had been allocated, at the wk12 kickoff, a narrow computer-vision role: train a YOLOv8n dummy detector and hand it to the flight team. **1a.** Everything around that core — the simulation framework, the progressive test ladder, the geofence logic, the blueprint system, the complaint letter to the course organiser, even Demetro’s FDR speaking scripts — I took on implicitly, one commit at a time, whenever I saw something between the team and a working demo. I initially framed this drift as conscientiousness (Hatton & Smith, 1995). In hindsight I now see it was partly avoidance: building infrastructure alone was a way of feeling productive during the roughly eight weeks when three cancelled flight days left the team with no hardware to iterate against, and it protected me from the harder Western-team-engineering skill of slowing down so that a teammate could catch up. My Ukrainian training told me one person delivers; the project told me that is a failure mode dressed as a strength.

1b. The contribution I am most proud of sits inside that same uncomfortable truth. It is not the retrained model at $mAP_{50} = 0.995$, nor the 820-plus commits, nor the D6 Goldmine climb from 74 to 85.2 across six scoring cycles. It is the fact that on the first cancelled flight day (2026-03-11), when the weather killed our only window, I did not go home — I wrote `FIELD_QUICK_REF.md` for no-internet use, calibrated the focal length to 5.46 mm against a tape measure at 1 m, ran the checkerboard lens calibration, and closed ten commits before dark. That day crystallised my one-line thesis about this project, which I will defend: *the simulation framework we built ourselves is the only reason this project produced any verifiable engineering output at all*. What I am proud of is not the output — it is that I refused to let a missing drone become a missing report. The counterfactual is concrete: had I treated the field days as the unit of work, the team would have delivered zero flights *and* zero verifiable data. Instead we delivered zero flights and a working state machine, a 2508-line simulator with GPS drift and camera shake, a 146-test pytest harness, and a web ground station a pilot could operate unaided. Had I acted on the “build a simulator first” instinct a week earlier than I did, we would have saved roughly five days of idle waiting — a pattern I now recognise in myself: I trust the escape route too late.

1c. What this experience has forced me to reframe, and what I will focus on differently in future, is the distinction between *capability* and *enablement*. I leave this project believing that engineering leadership is less about being the one who can fix the geofence sign-convention bug at midnight (reflection-in-action, Schön, 1983), and more about the skill of making my fix legible enough that the teammate who owns the module would have found it without me. The evidence that convinced me is the `passive_watch_2.py` handoff to Robin (commit `a5b1ab7`): I built the `-robin` flag and the `cv_mode` polling interface specifically so his state machine could consume my vision output over HTTP without touching my code — and Robin then integrated for two days without asking me a single question. That was the first teamwork move I made on this project that scaled without my presence, and it happened in week 20, not week 12. In my next team project — I start an internship with a robotics group in June — I will schedule two hours per week of explicit pair-programming and handoff-document review, and the falsifiable signal I will watch for is whether a teammate can complete a handoff task independently within 48 hours of receiving it. If they cannot, my documentation has failed, not their attention. That is a sharper test than “communicate better,” and it is the test I should have been running on myself since wk12.

My team

2a. Structure — discussed once, drifted twice, re-opened under pressure. At the wk12 kickoff we did hold an explicit structure conversation, and I want to credit Robin for forcing it: I would have skipped it. We split on module ownership — Robin took the flight loop and state execution, I took perception and simulation, Edward took the Cube and hardware interface, Demetro took client-facing communication and the FDR presentation, and our PM coordinated schedule and budget. We deliberately declined to name a single technical lead because the brief (R08) devolves autonomy-level choices to the team and we wanted that authority distributed. The structure then drifted twice. The first drift was silent: around wk16, as hardware kept slipping, the project’s centre of gravity migrated from the bench to my laptop because that was the only surface where forward progress was possible — nobody declared it, it just happened. The second drift was loud: at wk17, after I caught Robin and myself building overlapping state machines in parallel, I asked for a renegotiation, and we agreed in writing (`docs/DESIGN_DECISIONS.md:107–109`) that his script would own the flight loop and call my `passive_watch` as an HTTP service for detections. Looking back I now see we optimised for task division over communication cadence because task division felt unambiguous and cadence felt like overhead — and the first silent drift happened precisely because we had no routine for restating structure when the environment shifted. I did not understand before this project that unexplicit structure is a luxury which only survives in slack-rich teams.

2b. What worked (and the wk17 FSM conflict I had to manage). The most impactful team-working element, measured by what it unblocked, was a handoff ratio I had not previously noticed: the four hours I spent writing `docs/VISION_PIPELINE_FOR_COLLEAGUE.md` and the `-robin` integration flag (commit `a5b1ab7`) bought Robin roughly twenty hours of unblocked integration time across wk20 — he wired `passive_watch` into his mission script without asking me a single clarifying question for two days. I now understand those docs worked because they traded my solo time for Robin’s onboarding time at roughly 5:1, better team economics than I had assumed documentation ever delivered; I had previously treated handoff docs as a cost I paid for politeness.

The conflict I want to name happened in wk17. I had proposed an 11-state FSM for `main.py` with explicit transitions, timeout guards, and a geofence repulsor; Robin and Demetro pushed back in the same Teams thread, Robin arguing that a two-out-of-five maintainability floor meant the FSM was over-engineered, Demetro arguing the extra states would not survive a hardware-debug session with Edward. My first reaction was defensive — I genuinely believed the FSM was safer given our NFZ edge cases, and commit `ce5c036` later proved at least part of that belief correct when the RTL mode tuple fix needed an explicit state transition to `LANDING`. But I took 24 hours (reflection-in-action, Schön, 1983), reread Robin’s flight script, and realised half of my insistence was that the FSM was my code. I proposed a compromise on the wk17 call: I would keep the full FSM for the autonomous path in `main.py`, but strip it to a lightweight polling structure exposed over HTTP for Robin’s script — that is what the `-robin` flag in `passive_watch_2.py` actually is, not a feature but an interface concession. Robin accepted, Demetro signed off, and the wk20 integration took an afternoon instead of a week. The Hatton-level move I now see is that I had been conflating “is this the right design” with “is this the right design *for this team*,” and

these two kinds of rightness can come apart. I was right about the FSM and wrong about the context. A second, quieter conflict was the wk20 workload-and-comms tension with our PM after three flight days had been cancelled. I felt the team was drifting into silent resignation. Instead of confronting him directly — something I historically do badly — I drafted a complaint letter in measured factual prose, iterated the tone six times (commits 26e3255, f3401e4, d946838, 5af87d0, 847518e, 4b0fd91), and shared it only after Robin cut three sentences that were “technically accurate but personally sharp.” The PM read it, disagreed with one paragraph, we rewrote that paragraph together, and the letter went out with all five names. The mediating action was *externalising an internal conflict* — converting a brewing interpersonal frustration into a shared document with a shared audience. I had not previously noticed in myself that I can only handle confrontation when it is mediated by writing, and I now see this is a pattern across my personal life, not just this project.

2c. Even better if — three concrete changes, with falsifiable signals. First, I would institute a weekly one-line decision log from wk1, not wk12 — every call that bound more than one person’s work, with date, owner, and reason. The silent wk16 drift would have been caught by any written record; the signal that it was working would be: zero overlapping-work surprises at integration checkpoints. Second, I would front-load interface contracts *before* parallel work begins, specifically a two-hour whiteboard session in the first week of any new module to agree inputs, outputs, and failure modes at every seam — a single wk14 session on the `passive_watch` HTTP contract would have removed the whole wk17 conflict. Third, I would stop treating documentation as a substitute for conversation. The two pieces of writing that actually changed teammates’ behaviour in this project were the 400-word pivot message and the 2000-word complaint letter — both worked because they were short, timely, and accompanied by a live call. The 3000-word colleague handoff doc, by contrast, sat unread until Robin finally needed it. The falsifiable signal for next time: any document longer than 500 words must be paired with a 15-minute walkthrough inside a week of being written, or I will treat it as unsent.

My impact

3a. How my contributions drove the team forward. The honest thesis of this section is the one I wrote into our group complaint letter: *the simulation framework we built ourselves is the only reason this project produced any verifiable engineering output at all*. I want to defend that claim with mechanism, not assertion, because it is exactly the kind of line that at Merit-level reads as self-flattery.

The anchor is a full Kolb (1984) cycle, which I used deliberately rather than retrospectively. My *concrete experience* was three collapsed flight days: the 2026-03-11 weather cancellation, the 2026-03-30 hardware session that produced no airborne minutes, and a third day that ended with our pilot standing on a wet field while Edward and I debugged a corrupt `calibration_data.npz`. After the first cancellation my *reflective observation* was harder than “we lost a day”: the team’s entire definition of “progress” had silently assumed hardware access we did not have, and the rest of the team was idle because their planned work needed a drone in the air. I was blocked too, but I had a laptop. That asymmetry was the data point. My *abstract conceptualisation* was the move I am most proud of: if `vision.py` exposed a single interface (`detect_in_image` returning `(found, x, y, conf)`) and auto-selected between Ultralytics on laptop and TFLite on Pi, and if `config.py` auto-detected its hardware, then the same state machine, geofence, and search pattern could run in both places — anything that ran in simulation would, in principle, run on hardware. My *active experimentation* was six weeks of hammering on that premise: `simple_simulator.py` grew to 2508 lines with GPS drift and camera shake; `main.py` was refactored 1411 → 754 lines across six modules with 146 pytest tests; and the `tests/flight/0a-4` ladder plus 127 unit tests gave me a way to know, before any field day, which scripts could survive contact with hardware. When we finally ran on the Pi the delta was thin — a bbox-coordinate bug (commit 3d62e6a), a double-scaling fix (7eb7cc8) and a deleted calibration file — and the system came up. I now see the simulation framework was not a backup plan but a *statement*: verifiable engineering is possible without hardware access, and simulation-first is no longer a personal preference for me but a claim about what the discipline owes.

Three other contributions matter here and I mention them compactly. I unblocked Robin by building him a dedicated HTTP integration path (`passive_watch_2.py`, commit a5b1ab7) with a JSON schema and `-smart-clear` flag. I unblocked Demetro by building his two FDR slides from scratch and writing 465 words of speaking script across 4–5 iterations. And I drove the group’s external communication about

the hardware failures via the six-iteration complaint letter the whole team signed.

Methods evaluated against observable behaviour change per author-hour. I used six comms methods with named audiences during this project: (1) a 4-hour `VISION_PIPELINE_FOR_COLLEAGUE.md` doc for Robin; (2) the `-robin` JSON/HTTP interface for Robin’s mission script; (3) FDR slides and speaking scripts for Demetro (and the FDR panel); (4) the browser ground-station for the pilot; (5) the clarification letter for the course organiser; and (6) a 400-word Teams pivot message for the team. Against the criterion *did a teammate change behaviour within 48 hours*, the two fastest were the pivot message (team committed to simulation-first the same night) and the `-robin` flag (Robin integrated unaided for two days). The 3000-word handoff doc, by contrast, sat unread until Robin finally needed it in wk20. The lesson, which I did not expect, is that prose quality was almost orthogonal to behaviour change — timing, pairing, and interface shape dominated.

But the sober closing of 3a is that the simulation framework could *not* tell us what it could not tell us. It could not predict motion blur at 5 m/s with a real IMX296. It could not find the descent stall we accepted as a SITL artefact. It could not tell us what GPS timing lag would do to a multi-pass lock — that came out of DJI video analysis, after the fact. The statement my simulation made is true, and the limits of that statement are also true: sim-to-real parity is an argument I can only win about the things I modelled, and I was disciplined about that only because I had been forced to be.

3b. What I would add or re-focus on in future. I mistook technical leadership for team leadership. I now see they are orthogonal — you can build great tools that do not enable anyone — and in this project the gap showed up as a metric: the number of times a teammate other than me edited `simple_simulator.py` or `main.py` was zero. Three concrete mechanisms, each with a falsifiability test, follow. First, I will shift from “building tools I need” to “building tools the team needs,” and each new tool must have at least one named teammate beneficiary documented *before* it ships — if I cannot name them, I do not start. Second, no file larger than 500 lines will be refactored solo: I will pair on the review, and the observable signal is whether the pair partner can edit that file independently within 48 hours of the walkthrough. Third, I will schedule two hours per week of explicit teaching sessions, graded not by whether I explained something but by whether the teammate completes a handoff task within 48 hours without asking me a second question. If after four weeks the signals are not firing, the mechanism must change, not my reassurance that I meant well.

My support

4a. What others did that helped me work more effectively. Four people changed how I worked, and I want to name the specific mechanism for each because a generic list of thanks is a Merit-ceiling move.

Edward is the reason the Cube ever spoke to the Pi. Around wk15 I had spent most of a weekend trying to make `pymavlink` read from `/dev/ttyAMA0` on Python 3.13, and it was Edward — after I had filed three dead-end notes in Teams — who pointed out that `pyserial` reads are broken on Python 3.13 and that the workaround was a `mavproxy` UDP bridge. That 40-minute conversation unlocked roughly two weeks of Pi testing. The uncomfortable realisation was not the technical fact; it was that my instinct to “finish debugging before asking for help” is a cost I pay, not a professionalism I exhibit. Edward’s throwaway remark that “just because the test passes on laptop doesn’t mean the hardware does” reframed how I thought about simulation fidelity for the rest of the project — I stopped treating SITL parity as evidence and started treating it as a hypothesis.

Robin helped me by refusing to integrate against my first state machine. In wk17 I had a design I believed was technically correct and he pushed back until I could prove the extra states caused materially different decisions; his pushback is the reason the `main.py` refactor (1411 → 754 lines, 6 modules) happened at all. What I had framed as a technical disagreement was actually a maintainability disagreement, and I preferred the technical framing because it protected me from noticing I had built something only I could read. **Demetro** accepted drafts written in my voice without turning collaboration into a status contest. **Our pilot** operated the browser ground station unaided during the field-day dry runs, validating the minimalist button layout in a way no laptop testing could have. **Sid** delivered the single DJI flight video with SRT telemetry that anchored FOV calibration, 300 synthetic backgrounds, and the retrained model at $mAP50 = 0.995$.

4b. What I did that helped others work more effectively. I will put three specific feedback episodes on record, because “I gave constructive feedback when needed” is the single most common Merit-ceiling sentence in UK engineering reflection.

Feedback to Demetro — FDR speaking scripts. When Demetro asked for help with his two FDR slides (commit 32fb0c5) I built the layouts and wrote full speaking scripts (246 + 219 words, 3.1 min total, commits b2a9cd4, ba667f2), iterating 4–5 times. His first reaction was polite gratitude, but on the third pass he pushed back: the wording was not how he spoke. The observable behaviour change came on FDR day — he delivered the two slides confidently, on time, in his own cadence, and a week later when he prepared his next set of slides himself he was noticeably using specific numbers rather than adjectives (a habit I had modelled in draft three). What I learned about my own delivery is harder than what he learned from mine: my first three drafts were in my register, not his, and I now see that good feedback to a peer presenter means writing in their voice, not polishing yours. The scarce resource was register, not prose quality.

Feedback to Robin — CENTERING timeout and the handoff contract. In wk19 I noticed Robin’s flight script had no timeout guard on the `CENTERING` state: if the target was lost mid-descent the loop would hang on a detection that never came. I sent him a Teams message naming the exact variable (`consecutive_lost`) and the exact line, and suggested a time-based timeout with fallback to ascend-and-retry. He fixed it the same afternoon — the fix is now visible in `4_detect_and_center.py`. The behaviour change that mattered came a week later: in wk20 he pulled me aside to say he had caught a similar timeout bug in a separate module because he was now “looking for that shape of bug.” That is the outcome I wanted — not the specific patch, but the shape of the check propagating into his working model. The generalisable lesson is that when giving feedback to a high-performing peer, specificity is the scarce resource, not softness.

Feedback to the pilot — ground-station button layout and jargon removal. When I showed the pilot the first `pi_flight.py` dashboard he said the button grid was too dense to hit reliably while holding an RC transmitter. I proposed a minimal four-button set (Y / N / M / L) mapped to the RC thumb-reach pattern, and wired an emergency RTL into the override guard so mode 9 (LAND) from the RC switch would be honoured. The subtler adaptation was lexical. My first draft of his pre-flight card used phrases like “geofence repulsor active within 10m buffer” and “MAV_CMD_NAV_TAKEOFF ACK race.” After watching him hesitate over the first line, I rewrote them as “the drone pushes itself away from the red fence if it gets within 10m” and “the drone sometimes doesn’t lift off because it is waiting for a confirmation message that never comes.” The rewritten card was read aloud in one pass without a pause. During the field-day dry runs the pilot then triggered the abort three times without looking at the screen, and on the same day asked for a similarly minimal layout on the `VERIFY` confirmation screen — the moment my frame shifted from designing for me-as-operator to designing for the pilot who does not have to notice the UI.

The non-feedback infrastructure I built was the quieter half of this work: `brain-dump.md`, `MEMORY.md`, the 500-line blueprint system, `CLAUDE.md` as project ground truth, and `FIELD_QUICK_REF.md` for no-internet field use. I also drafted the clarification letter to the course organiser — the writing that taught me most about register, because my first three drafts were in my voice and my anger, and the fourth was in the team’s voice only after Robin cut three sentences that were “technically accurate but personally sharp.”

What I leave this project believing. I came as a Ukrainian-trained engineer whose reflex under failure was “one person delivers.” I leave believing its near-inversion: *the engineer who delivers alone is one recruitment away from being the single point of failure in whatever system she builds*. The simulation framework proved I could carry an entire project’s verifiable output on my laptop for eight weeks; the modularity audit, the wk17 FSM conflict, and Robin’s wk20 unaided handoff each told me, in different registers, that the same act which delivered the report was also the act that kept my teammates standing at the edge of their own project. My falsifiable internship signal: in my first six weeks, does at least one teammate independently modify a module I own without asking permission first? If no, I carried the old reflex forward and this project’s lesson has not yet landed. If yes — even once — I have started to become an engineer whose work enables other people’s work, which is the only kind of engineering I now think is worth aspiring to.